

SDR-SLC

SLC-DISC Service Discovery Specification

Field	Value
Document	SDR-SLC-DISC-1.0
Title	SLC-DISC Service Discovery Specification
Version	1.0 (Protocol) Release 1.1 (revised for implementers)
Status	RELEASE CANDIDATE
Date	May 2026
Author	Ivo van Ling, PA2IX
Organisation	IXChange BV
Licence	Creative Commons CC BY 4.0 — Open Source

Table of Contents

Table of Contents	2
Revision History	4
1. Introduction	5
2. Normative Language	6
3. Terminology	7
4. Implementing SLC-DISC from Scratch	8
4.1 What SLC-DISC Actually Does	8
4.2 On STM32 with lwIP	8
4.3 On Raspberry Pi (Python daemon)	9
4.4 The Instance Name	9
4.5 When to Send the Announcement	10
5. Service Advertisement Parameters	11
6. TXT Record Fields	12
6.1 Mandatory Fields	12
6.2 Optional Fields	12
6.3 Complete TXT Record Example	13
7. Announcement Behaviour	14
7.1 On Link-Up	14
7.2 Responding to Queries	14
7.3 Clean Shutdown (Goodbye Packets)	14
8. IGMP and Multicast Configuration	15
8.1 Required Multicast Groups	15
8.2 STM32 MAC Filter Configuration	15
9. Platform Interoperability	16
9.1 macOS / Bonjour	16
9.2 Linux / Avahi	16
9.3 Windows	16
9.4 Known Interoperability Issues	16
10. Verification Procedures	17
10.1 Browse Test (macOS)	17
10.2 Resolve Test (macOS)	17
10.3 Address Resolution Test	17
10.4 TCP Connection Test	17
10.5 Packet Capture Verification	18
10.6 Avahi Browse Test (Linux)	18
11. Conformance Requirements	19
11.1 Device	19

11.2 Client.....	19
12. Troubleshooting Guide	20
13. Security Considerations.....	21
13.1 mDNS Advertisement Spoofing	21
13.2 Future Authentication.....	21
14. References	22

Revision History

Rev	Date	Author	Description
1.0	2026-05-01	I. van Ling	Initial public draft.
1.1	2026-05-23	I. van Ling	Revised for implementers. Added §4 'Implementing SLC-DISC from Scratch' with step-by-step guidance for STM32/lwIP and Raspberry Pi daemon implementations. Added §6 detailed TXT record implementation notes. Expanded §9 IGMP section with STM32 register-level guidance. Expanded verification section with full expected outputs. Added troubleshooting guide §12. Aligned editorial style with SDR-SLC-CP-1.0-v2.

1. Introduction

SLC-DISC is the service discovery layer of the SDR-SLC protocol suite. Its job is simple: allow an SDR-SLC client to find SDR-SLC hardware on the local network automatically, without any manual IP address configuration, without a central server, and without any setup beyond plugging in the Ethernet cable.

SLC-DISC is built on two IETF standards that are natively supported on every platform that SDR-SLC targets. Multicast DNS (mDNS, RFC 6762) handles zero-configuration hostname resolution on the .local. domain. DNS-Based Service Discovery (DNS-SD, RFC 6763) adds structured service metadata that allows clients to learn device identity, frequency range, firmware version, and hardware role before opening a control connection.

On macOS, this is Apple's Bonjour. On Linux, it is Avahi. On Windows 10 and later, it is built into the OS. None of these require any installation or configuration from the user. Plug in the hardware, and it appears in the client's device list.

This document specifies what the device must advertise, how it must behave on the network, and how a client finds and verifies it. It also provides a practical implementation guide for developers building their first SLC-DISC implementation.

2. Normative Language

Keyword	Meaning
SHALL	Absolute requirement. Non-conformant if omitted.
SHALL NOT	Absolute prohibition.
SHOULD	Strong recommendation.
MAY	Optional.

3. Terminology

Term	Definition
SLC-DISC	Service discovery layer. The complete mDNS/DNS-SD implementation on the device.
mDNS	Multicast DNS (RFC 6762). Resolves .local. hostnames without a central DNS server.
DNS-SD	DNS-Based Service Discovery (RFC 6763). Advertises and browses services by type.
PTR record	DNS Pointer record. Maps a service type to an instance name.
SRV record	DNS Service record. Maps an instance name to a hostname and port number.
TXT record	DNS Text record. Carries key=value metadata about a service instance.
A record	DNS Address record. Maps a hostname to an IPv4 address.
TTL	Time To Live. Seconds for which a DNS record may be cached by clients.
Instance name	The mDNS name identifying this specific device. Format: SLC-XXYYZZ.
Bonjour	Apple's mDNS/DNS-SD implementation. Native on macOS and iOS.
Avahi	Open-source mDNS/DNS-SD implementation for Linux.
IGMP	Internet Group Management Protocol. Used to join IPv4 multicast groups.
224.0.0.251	The mDNS IPv4 multicast group address.
5353	The mDNS UDP port.

4. Implementing SLC-DISC from Scratch

This chapter is for the developer implementing SLC-DISC for the first time on either an STM32 microcontroller or a Raspberry Pi daemon. It explains exactly what you need to do, in what order, with concrete code guidance.

4.1 What SLC-DISC Actually Does

From the hardware device's perspective, SLC-DISC does three things:

- On Ethernet link-up: join the mDNS multicast group 224.0.0.251 via IGMP, then send an unsolicited mDNS announcement advertising the device's presence.
- When a client sends a DNS query for `_sdr-slc._tcp.local.`: respond with PTR, SRV, TXT, and A records describing the device.
- On clean shutdown: re-announce all records with TTL=0 (goodbye packets) to immediately remove the device from client caches.

That is all. No server, no registration, no configuration. The device is its own DNS server for its own name.

4.2 On STM32 with lwIP

The recommended approach on STM32 is to use the mDNS module included in lwIP 2.1+ (`mdns.c`). It is not compiled by default; you must enable it in `lwipopts.h`.

```
// In lwipopts.h:
#define LWIP_MDNS_RESPONDER 1
#define MDNS_MAX_SERVICES 2 // at minimum 1 for _sdr-slc._tcp
#define LWIP_IGMP 1 // required for mDNS multicast
#define LWIP_DNS 1 // required by mdns.c

// In your network initialisation (after netif add):
mdns_resp_init();
mdns_resp_add_netif(&gnetif, "SLC-000012", 120); // hostname, TTL

// Register the _sdr-slc._tcp service:
mdns_resp_add_service(&gnetif,
    "SLC-000012", // instance name
    "sdr-slc", // service type
    "tcp", // protocol
    4620, // port
    slcdisc_txt_cb, // TXT record callback
    NULL);

// The TXT record callback:
static void slcdisc_txt_cb(struct mdns_service *service, void *arg) {
    mdns_resp_add_service_txtitem(service, "proto=sdr-slc", 13);
    mdns_resp_add_service_txtitem(service, "proto ver=1.0", 12);
    mdns_resp_add_service_txtitem(service, "name=SLC-000012", 14);
    mdns_resp_add_service_txtitem(service, "manufacturer=IXChange BV", 22);
    mdns_resp_add_service_txtitem(service, "hw=SLC-TAYLOE-1.0", 17);
    mdns_resp_add_service_txtitem(service, "fw=1.0.0", 8);
    mdns_resp_add_service_txtitem(service, "role=rx", 7);
    mdns_resp_add_service_txtitem(service, "rx=1", 4);
    mdns_resp_add_service_txtitem(service, "tx=0", 4);
    mdns_resp_add_service_txtitem(service, "minhz=1800000", 13);
    mdns_resp_add_service_txtitem(service, "maxhz=30000000", 14);
}
```

TIP

The lwIP mDNS module handles the announcement, query responses, and goodbye packets automatically once you call `mdns_resp_add_netif()` and `mdns_resp_add_service()`. You do not need to implement UDP multicast send/receive yourself. The module also handles the required conflict detection (probing) before announcing.

NOTE

The hostname you pass to `mdns_resp_add_netif()` becomes the mDNS hostname (e.g. SLC-000012.local.). Generate it from the last 3 octets of your Ethernet MAC address in uppercase hex. This guarantees uniqueness on the LAN without any configuration.

4.3 On Raspberry Pi (Python daemon)

For a Raspberry Pi daemon bridging an RTL-SDR or other hardware, use the zeroconf Python library. It handles all the mDNS/DNS-SD complexity.

```
from zeroconf import Zeroconf, ServiceInfo
import socket

def get_local_ip():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.connect(("8.8.8.8", 80))
    ip = s.getsockname()[0]
    s.close()
    return ip

# Get device MAC address (last 3 octets for instance name)
import uuid
mac = uuid.getnode()
mac_suffix = "%06X" % (mac & 0xFFFFFF)
instance_name = f"SLC-{mac_suffix}"

zeroconf = Zeroconf()

info = ServiceInfo(
    "sdr-slc.tcp.local.",
    f"{instance_name}.sdr-slc.tcp.local.",
    addresses=[socket.inet_aton(get_local_ip())],
    port=4620,
    properties={
        "proto": "sdr-slc",
        "proto ver": "1.0",
        "name": instance_name,
        "manufacturer": "PA2IX",
        "hw": "RTL-SDR-Bridge-1.0",
        "fw": "1.0.0",
        "role": "rx",
        "rx": "1",
        "tx": "0",
        "minhz": "100000",
        "maxhz": "1766000000",
    },
    server=f"{instance_name}.local.",
)

zeroconf.register_service(info)
print(f"Announced {instance_name} on mDNS")

# On shutdown:
# zeroconf.unregister_service(info) # sends goodbye packets
# zeroconf.close()
```

TIP

The zeroconf library handles everything: multicast group join, announcement, conflict detection, query responses, and goodbye packets. Install it with: `pip3 install zeroconf`. It runs on Python 3.8+ and works on Raspberry Pi OS, Ubuntu, and macOS.

4.4 The Instance Name

The instance name is the unique identifier for your device on the mDNS network. The format is SLC-XXYYZZ where XXYYZZ are the last three octets of the Ethernet MAC address in uppercase hexadecimal.

```
// STM32: derive instance name from MAC address
// Assuming HAL_ETH_GetMACAddress() fills mac[6]:
uint8_t mac[6];
HAL_ETH_GetMACAddress(&heth, mac);
char instance_name[16];
snprintf(instance_name, sizeof(instance_name),
         "SLC-%02X%02X%02X",
         mac[3], mac[4], mac[5]);
// Result: e.g. "SLC-12AB34"

// This name is used as:
// mDNS hostname:      SLC-12AB34.local.
// mDNS instance name: SLC-12AB34.sdr-slc.tcp.local.
// TXT name field:     name=SLC-12AB34
```

NOTE

Using the MAC address guarantees uniqueness without any configuration. Two SDR-SLC devices from the same manufacturer will have different MAC addresses and therefore different instance names. If the user wants a human-friendly name, they can set it via a configuration interface, but the MAC-derived default must work out of the box.

4.5 When to Send the Announcement

The announcement sequence on link-up must be ordered correctly, especially on managed switches with IGMP snooping:

Link-up sequence (timing critical on managed switches):

```
t = 0 ms    Ethernet link established
t = 1 ms    IGMPv2 Membership Report sent to 224.0.0.251
             (join the mDNS multicast group)
t = 100 ms  Wait for switch IGMP table to update
             (switch now forwards mDNS packets to this port)
t = 100 ms  mDNS announcement sent:
             PTR _sdr-slc._tcp.local. → SLC-XXYYZZ._sdr-slc._tcp.local.
             SRV SLC-XXYYZZ._sdr-slc._tcp.local. :4620
             TXT SLC-XXYYZZ._sdr-slc._tcp.local. [key=value pairs]
             A   SLC-XXYYZZ.local. → [device IP]
```

CRITICAL: The IGMP report MUST be sent before the announcement.
If the switch has not learned the multicast membership,
the announcement will not be forwarded to client ports.

WARNING

The 100 ms wait between the IGMP report and the mDNS announcement is not optional on managed switches. Many enterprise and prosumer switches need time to process the IGMP report and update their forwarding tables. Skipping the wait causes the announcement to be dropped, and the device will not be discovered until the client sends a query.

5. Service Advertisement Parameters

Parameter	Value
Service type	_sdr-slc._tcp.local.
Transport	TCP
Default control port	4620
mDNS multicast group	224.0.0.251 (IPv4), ff02::fb (IPv6)
mDNS port	5353/UDP
Record TTL	120 seconds (per RFC 6762 §11.3)
Instance name format	SLC-XXYYZZ (last 3 octets of MAC address, uppercase hex)
Example hostname	SLC-000012.local.
Example instance	SLC-000012._sdr-slc._tcp.local.

6. TXT Record Fields

DNS-SD TXT records carry structured metadata about the service. The client reads these fields before opening a TCP control connection, allowing it to verify compatibility and display device identity to the operator.

6.1 Mandatory Fields

Every SDR-SLC device SHALL include all of these fields. A client that finds a service instance missing any mandatory field SHOULD treat it as non-conformant and not connect.

Field	Example value	Description
proto	sdr-slc	Protocol family identifier. Always the literal string "sdr-slc". Allows clients to distinguish SDR-SLC services from other _tcp services on the same network.
proto_ver	1.0	Protocol suite version. Clients SHOULD check this for compatibility. A client that does not support the advertised version SHOULD display a warning.
name	SLC-000012	Human-readable device name. Same as the mDNS instance name by default. MAY be overridden by user configuration.
manufacturer	IXChange BV	Hardware manufacturer name. For IXChange hardware, always "IXChange BV". For third-party hardware, use your own name.
hw	SLC-TAYLOE-1.0	Hardware revision string. Manufacturer-defined. Useful for bug tracking and support.
fw	1.0.0	Firmware version string. Use semantic versioning (MAJOR.MINOR.PATCH).
role	rx	Primary device role: "rx", "tx", or "rxtx".
rx	1	"1" if receive is implemented. "0" otherwise.
tx	0	"1" if transmit is implemented. "0" otherwise.

6.2 Optional Fields

Field	Example value	Description
model	SDR-SLC HF Transceiver V1.0	Full hardware model identifier for display in the client device list.
stream	vita-49-slc	Streaming protocol identifier. Informational; actual negotiation happens in SLC-CP.
minhz	1800000	Minimum receivable frequency in Hz. Decimal string. SHALL be parsed as 64-bit unsigned integer by clients. Allows clients to show frequency range in the device list before connecting.
maxhz	30000000	Maximum receivable frequency in Hz. Same rules as minhz.

WARNING *minhz and maxhz are DNS TXT string values containing decimal integers. Clients SHALL parse them as 64-bit unsigned integers. A 32-bit integer overflows at approximately 4.295 GHz. Hardware supporting frequencies above 4.3 GHz MUST express these values as full decimal strings (e.g. "10000000000" for 10 GHz).*

6.3 Complete TXT Record Example

The following shows the complete DNS record set for a device with instance name SLC-000012 at IP 192.168.5.35 on port 4620:

```
; PTR: service browse answer
_sdr-slc._tcp.local. PTR SLC-000012._sdr-slc._tcp.local.

; DNS-SD meta-query: service type enumeration
_services._dns-sd._udp.local. PTR _sdr-slc._tcp.local.

; SRV: hostname and port
SLC-000012._sdr-slc._tcp.local. SRV 0 0 4620 SLC-000012.local.

; TXT: metadata
SLC-000012._sdr-slc._tcp.local. TXT
    proto=sdr-slc
    proto ver=1.0
    name=SLC-000012
    manufacturer=IXChange BV
    hw=SLC-TAYLOE-1.0
    fw=1.0.0
    role=rxtx
    rx=1
    tx=1
    model=SDR-SLC HF Transceiver V1.0
    stream=vita-49-slc
    minhz=1800000
    maxhz=30000000

; A: hostname resolution
SLC-000012.local. A 192.168.5.35
```

7. Announcement Behaviour

7.1 On Link-Up

- Send IGMPv2 Membership Report for 224.0.0.251.
- Wait at least 100 ms.
- Send unsolicited mDNS announcement (all four record types: PTR, SRV, TXT, A).

7.2 Responding to Queries

Query type	Records returned
PTR for _sdr-slc._tcp.local.	PTR + SRV + TXT + A (the full set)
PTR for _services._dns-sd._udp.local.	PTR for _sdr-slc._tcp.local.
SRV for instance name	SRV + A
TXT for instance name	TXT
A for hostname (SLC-XXYYZZ.local.)	A

Responses SHALL be sent to the mDNS multicast group (224.0.0.251:5353) with the QR bit set (response). Known-answer suppression per RFC 6762 §7 SHOULD be implemented to avoid redundant responses.

7.3 Clean Shutdown (Goodbye Packets)

On clean shutdown the device SHOULD re-announce all DNS records with TTL=0. Clients that receive a goodbye packet remove the service from their cache immediately, preventing the client from showing a stale entry for the TTL duration (120 seconds). If power is abruptly removed, clients will purge the entry when the TTL expires.

8. IGMP and Multicast Configuration

8.1 Required Multicast Groups

Group	IPv4 address	MAC address	Purpose
mDNS	224.0.0.251	01:00:5E:00:00:FB	SLC-DISC announcement and query response.
PTPv2 (opt.)	224.0.1.129	01:1B:19:00:00:00	PTPv2 SYNC/FOLLOW_UP/ANNOUNCE (if implemented).

8.2 STM32 MAC Filter Configuration

The STM32H7 Ethernet MAC must be configured to receive multicast frames addressed to 01:00:5E:00:00:FB. This requires setting up the MAC multicast hash filter correctly.

```
// After calling HAL_ETH_Start() and lwIP igmp_joingroup():

// The STM32H7 ETH MAC uses a 64-bit hash filter for multicast.
// For 01:00:5E:00:00:FB (mDNS), the CRC32 of the address gives
// the hash table index. lwIP handles this via ETH_MACFilterConfig.

// Simplest approach: pass all multicast (for development/testing)
ETH_MACFilterConfigTypeDef filter = {0};
filter.PassAllMulticast = ENABLE;
HAL_ETH_SetMACFilterConfig(&heth, &filter);

// For production: use the hash filter and set only the required bits.
// The mDNS MAC 01:00:5E:00:00:FB hashes to bit 32 in the hash table.
// Consult the STM32H7 reference manual §57.8.3 for MACHT0R/MACHT1R.

// Also required: join the IGMP group via lwIP
ip4_addr_t mcast_addr;
IP4_ADDR(&mcast_addr, 224, 0, 0, 251);
igmp_joingroup(netif_ip4_addr(&gnetif), &mcast_addr);
```

NOTE

The HAL_ETH_SetMACFilterConfig() call that lwIP makes internally may overwrite your custom hash filter settings. If you find multicast reception stopping after lwIP runs, call your filter configuration after the lwIP network interface initialisation is complete.

9. Platform Interoperability

9.1 macOS / Bonjour

macOS includes native Bonjour mDNS/DNS-SD support. The SDR-SLC Swift client uses the Network framework or `dns_sd.h` API to browse for `_sdr-slc._tcp.local`. Discovery is zero-configuration. Bonjour caches DNS records for their TTL period. After a device goes offline without sending goodbye packets, macOS will attempt to reconfirm the record at 80% and 90% of TTL before expiring it, so the entry persists for up to 120 seconds after the device disconnects.

9.2 Linux / Avahi

Avahi is the reference implementation on Linux. The `avahi-browse` command-line tool is useful for testing.

```
# Browse for SDR-SLC services:
avahi-browse _sdr-slc. tcp

# With resolution (-r) shows hostname and TXT:
avahi-browse -r _sdr-slc._tcp

# Resolve a specific instance:
avahi-resolve -n SLC-000012.local.
```

9.3 Windows

Windows 10 version 1703+ includes a built-in mDNS client. Third-party clients may use the Bonjour SDK for Windows for DNS-SD browsing. The SDR-SLC client on Windows SHOULD use Windows DNS-SD APIs or the Bonjour SDK.

9.4 Known Interoperability Issues

Issue	Platforms affected	Mitigation
IGMP snooping without querier blocks mDNS multicast.	Managed enterprise switches.	Enable IGMP querier on the VLAN, or disable IGMP snooping for the VLAN.
mDNS disabled by MDM policy.	Windows in corporate domains.	Use static IP fallback; document the configuration procedure.
Avahi daemon not running.	Minimal Linux installations.	Install avahi-daemon: <code>sudo apt install avahi-daemon</code>
IPv6 mDNS (ff02::fb) not forwarded.	Some managed switches.	Use IPv4 multicast only; SDR-SLC uses IPv4 exclusively.
Firewall blocking UDP 5353.	Any platform with local firewall.	Allow UDP 5353 on the LAN interface.

10. Verification Procedures

Use these procedures to verify that your SLC-DISC implementation is working correctly. Run them from a macOS or Linux host on the same LAN as the device.

10.1 Browse Test (macOS)

This confirms the device is announcing itself and responding to PTR queries:

```
dns-sd -B _sdr-slc._tcp local.
```

Expected output within a few seconds:

```
Browsing for _sdr-slc._tcp.local.
```

Timestamp	A/R	Flags	if	Domain	Service Type	Instance Name
14:32:15.001	Add	2	14	local.	_sdr-slc._tcp	SLC-000012

If nothing appears within 10 seconds:

1. Check that the device Ethernet link is up (LED on switch port).
2. Check that the device is sending IGMPv2 reports (Wireshark: igmp).
3. Check that the mDNS announcement is being sent (Wireshark: udp port 5353).
4. Check IGMP snooping configuration on managed switch.

10.2 Resolve Test (macOS)

This confirms SRV and TXT records are correctly formed:

```
dns-sd -L "SLC-000012" _sdr-slc._tcp local.
```

Expected output:

```
Lookup SLC-000012._sdr-slc._tcp.local.
```

```
DATE: ---Mon 20 May 2026---
```

```
14:32:20.000 SLC-000012._sdr-slc._tcp.local. can be reached at
              SLC-000012.local.:4620 (interface 14)
proto=sdr-slc proto_ver=1.0 name=SLC-000012
manufacturer=IXChange BV hw=SLC-TAYLOE-1.0 fw=1.0.0
role=rxtx rx=1 tx=1
model=SDR-SLC HF Transceiver V1.0
minhz=1800000 maxhz=30000000
```

Verify: port is 4620, all mandatory TXT fields are present.

10.3 Address Resolution Test

```
dns-sd -G v4 SLC-000012.local.
```

Expected output:

```
DATE: ---Mon 20 May 2026---
```

```
14:32:25.000 SLC-000012.local. AAAA 192.168.5.35
```

```
ping -c 3 SLC-000012.local.
```

Expected: 3 successful ping replies from 192.168.5.35

10.4 TCP Connection Test

```
# Verify SLC-CP is listening on port 4620:
```

```
nc -z SLC-000012.local. 4620 && echo "TCP port 4620 open"
```

```
# Try a minimal hello exchange:
```

```
echo -n
```

```
'\x00\x00\x00\x4f{"type":"request","id":"t1","command":"hello","params":{"client_name":"test","client_version"
```

```
| nc SLC-000012.local. 4620 | hexdump -C | head -5
```

```
# The device should respond with a 4-byte length prefix + JSON response.
```

10.5 Packet Capture Verification

```
# Capture all mDNS traffic from the device:
tcpdump -i eth0 -n 'src 192.168.5.35 and udp port 5353'

# Capture IGMP from the device:
tcpdump -i eth0 -n 'igmp and src 192.168.5.35'
# You should see: 224.0.0.251 > 224.0.0.251: igmp v2 report

# Verify PTR record in mDNS response:
tcpdump -i eth0 -n 'udp port 5353' -A | grep sdr-slc
```

10.6 Avahi Browse Test (Linux)

```
avahi-browse -r _sdr-slc._tcp

Expected output:
+ eth0 IPv4 SLC-000012 _sdr-slc._tcp local
= eth0 IPv4 SLC-000012 _sdr-slc._tcp local
  hostname = [SLC-000012.local]
  address = [192.168.5.35]
  port = [4620]
  txt = [proto=sdr-slc proto_ver=1.0 name=SLC-000012 ...]
```

11. Conformance Requirements

11.1 Device

Requirement	Level
Advertise service type <code>_sdr-slc._tcp.local.</code>	SHALL
Use instance name derived from MAC address (SLC-XXYYZZ) by default	SHALL
Include all mandatory TXT record fields (§6.1)	SHALL
Send IGMPv2 Membership Report for 224.0.0.251 before mDNS announcement	SHALL
Wait at least 100 ms between IGMP report and mDNS announcement	SHOULD
Respond to PTR, SRV, TXT, and A queries	SHALL
Set record TTL to 120 seconds	SHALL
Send goodbye packets (TTL=0) on clean shutdown	SHOULD
Implement RFC 6762 known-answer suppression	SHOULD

11.2 Client

Requirement	Level
Browse <code>_sdr-slc._tcp.local.</code> before connecting to any device	SHALL
Verify <code>proto=sdr-slc</code> in TXT record before connecting	SHOULD
Parse <code>minhz</code> and <code>maxhz</code> as 64-bit unsigned integers	SHALL
Handle service loss gracefully (TTL expiry or goodbye packet)	SHALL
Retry connection with exponential backoff after connection failure	SHOULD
Display device name, manufacturer, and firmware version from TXT record	SHOULD

12. Troubleshooting Guide

Symptom	Most likely cause	How to check and fix
Device not discovered at all	IGMP report not sent, or mDNS announcement not sent.	Wireshark on switch port: look for IGMP and UDP port 5353 packets from the device IP within 200 ms of link-up.
Device discovered but immediately disappears	Goodbye packets being sent unintentionally, or TTL set to 0.	Wireshark: filter udp port 5353 and src device_ip. Look for packets with TTL=0 in the additional records.
Device discovered but TXT record missing fields	TXT callback not including all mandatory fields.	dns-sd -L on macOS and compare output against §6.1. Add missing fields to the TXT callback.
Device found on macOS but not Linux	IPv6 mDNS issue (ff02::fb not forwarded) while IPv4 works.	Check Avahi config: Avahi may prefer IPv6. Try avahi-browse -a to see all address families.
Device found on unmanaged switch but not managed	IGMP snooping blocking multicast without querier.	Enable IGMP querier on the switch VLAN, or temporarily disable IGMP snooping to confirm this is the cause.
Port 4620 shown in resolution but TCP connection refused	SLC-CP server not started, or bound to wrong interface.	telnet SLC-000012.local. 4620 and verify. On STM32: confirm lwIP TCP PCB is listening on port 4620 after netif_add.
minhz/maxhz shown incorrectly in client	Parsed as 32-bit integer, truncated.	Verify client uses uint64_t / BigInt for these fields. 30,000,000 is fine in 32-bit but 1,766,000,000 is close to the 32-bit limit.

13. Security Considerations

13.1 mDNS Advertisement Spoofing

mDNS advertisements are unauthenticated. Any host on the LAN can advertise a rogue `_sdr-slc._tcp.local.` service and redirect clients to an attacker-controlled host. Clients **SHOULD** verify the `proto` and `manufacturer` TXT record fields before connecting and **SHOULD** display device identity to the operator for confirmation.

13.2 Future Authentication

A future version of SLC-DISC **MAY** add device authentication at the TXT record level, for example a signed device certificate fingerprint that the client can verify against a trusted certificate authority. This is not required for version 1.0 which targets trusted LAN operation.

14. References

Reference	Title	Relevance
RFC 6762	Multicast DNS	mDNS protocol specification. Definitive reference.
RFC 6763	DNS-Based Service Discovery	DNS-SD protocol specification.
RFC 2782	DNS SRV records	SRV record format.
RFC 2119	Key Words for Use in RFCs	Normative keyword definitions.
SDR-SLC-CP-1.0	SLC-CP Control Plane	TCP control connection advertised by SLC-DISC.
SDR-SLC-VITA-1.0	VITA-SLC Stream Transport	I/Q stream transport.
SDR-SLC-PS-1.0	Protocol Suite Architecture	Overall architecture and design rationale.
zeroconf (Python)	zeroconf Python library	Recommended for Raspberry Pi daemon implementations.
lwIP mdns.c	lwIP mDNS module	Recommended for STM32/embedded implementations.